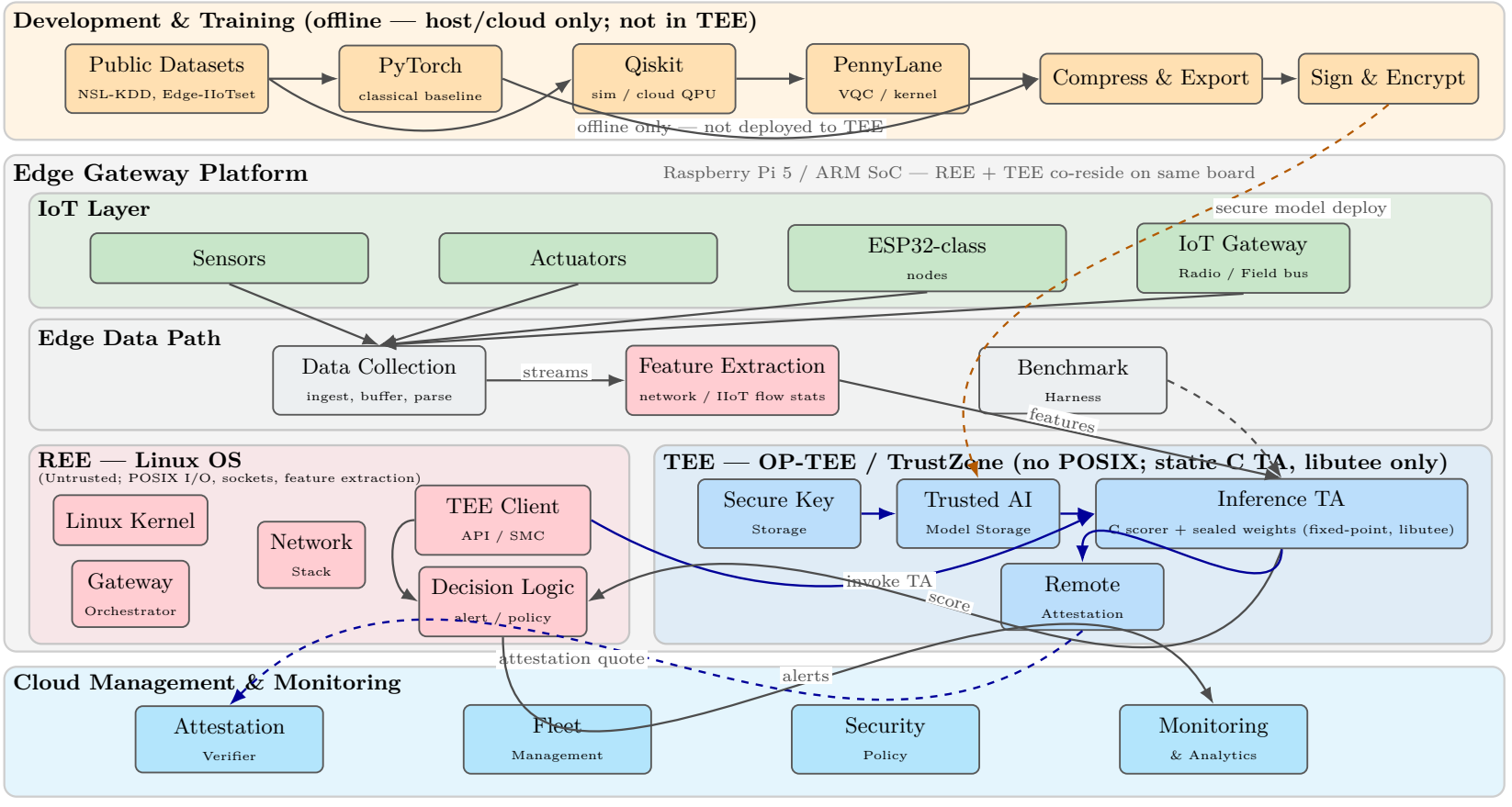
Trustworthy Hybrid Quantum–Classical AI for Secure IoT Edge Systems

Reference architecture (TRL 2–3) — Raspberry Pi 5 / ARM gateway; REE handles I/O, OP-TEE TA runs protected C inference only



Implementation Detail — Reviewer Q&A

Companion to system architecture figure (Page 1). Platform: Raspberry Pi 5 + OP-TEE, TRL 2–3.

1. Tell the implementation detail of the current system architecture

The architecture is a **three-phase, split-world** design:

Phase A — Offline (host/cloud, not on device): Public datasets (NSL-KDD, Edge-IIoTset, Bot-IoT) are used to train a PyTorch/TinyML classical baseline and hybrid QML models (Qiskit/PennyLane on simulator or cloud QPU). Models are compressed and exported as a **quantum-inspired C surrogate**, then signed and encrypted for deployment.

Phase B — Online REE (Linux on Pi 5): A Client Application (CA) using `libteec` performs all POSIX I/O: network capture/ingest, parsing, feature extraction, benchmark logging, and policy actions (alert/block/log). The Gateway Orchestrator coordinates the data path; the TEE Client API opens a session to the secure-world TA.

Phase B — Online TEE (OP-TEE Trusted Applications): A single **Inference TA** (static C, fixed-point, `libutee` only) loads sealed model weights from Trusted Model Storage, runs scoring, and returns an anomaly score via `TEE_InvokeCommand`. Separate logical TAs for Secure Key Storage and Remote Attestation may be merged in early prototypes. **Qiskit/PennyLane never run inside TrustZone.**

Phase C — Cloud: Attestation Verifier checks quotes; Fleet Management and Security Policy enforce device trust; Monitoring receives alerts.

Invoke sequence (CA → TA): `TEEC_InitializeContext` → `TEEC_OpenSession` → `TEEC_InvokeCommand(INFER, feature_vector[N])` → score → `TEEC_CloseSession`. Feature vector size N is fixed at build time (dataset-specific; e.g. 30–80 floats for NSL-KDD/Edge-IIoTset flows).

2. Do you assume the QAI works as Agent AI for IDS/IPS?

No. QAI is **not** an Agent AI and not an autonomous IDS/IPS agent.

It is a **trusted anomaly-scoring function** inside one Inference TA: input = fixed feature vector, output = numeric score (+ optional attestation evidence).

IDS-like: yes — the REE Decision Logic converts scores into alerts sent to cloud monitoring.
IPS-like: only if REE policy explicitly blocks or quarantines traffic based on the score — this enforcement runs in Linux (REE), not inside the TA.
There is no autonomous planning, tool use, LLM reasoning, or self-directed action loop in the secure world.

3. What data is used by the QAI (detection engine)?

The detection engine receives a **pre-computed feature vector** produced in the REE — not raw packets and not host audit logs.

Data sources align with public IoT security benchmark datasets:

- **NSL-KDD / Bot-IoT:** network flow statistics (duration, protocol, bytes, flags, service type).
- **Edge-IIoTset:** IIoT gateway telemetry — packet/header fields, protocol anomalies, sensor/gateway metadata aggregated into flow windows.

The REE Data Collection and Feature Extraction stages parse telemetry and normalize values into a fixed-size vector passed to the TA.

4. File access, system calls, network traffic — which apply?

Data source	Used?	Role in current architecture
Network traffic	**Yes (primary)**	Captured/parsed in REE; flow stats fed to feature extractor and then to TA.
System calls	No (out of scope)	Not collected; host HIDS is not the target use case.
File access	No (out of scope)	Not collected; may be added later as optional REE features, not in TA.

Scope: gateway-level IIoT/network anomaly detection, not endpoint host intrusion detection.

5. How much CPU and memory overhead do you assume?

All values are **measurement targets** on Raspberry Pi 5 (to be validated by the benchmark harness):

Metric	Assumed target
REE feature extraction latency	$\leq$ 5–10 ms per scoring window
TA inference latency	$\leq$ 10–50 ms per `InvokeCommand`
End-to-end latency (T)	$\leq$ 100 ms (ingest → alert)
REE CPU (steady state)	$\leq$ 10–15% of one Cortex-A76 core at ∼100 flows/s
TA CPU during inference	Burst only during `InvokeCommand`; no background loop in secure world
REE RAM (CA + features)	$\leq$ 32–64 MB working set for gateway daemon
TA heap (`TA_DATA_SIZE`)	256 KB–1 MB (Pi OP-TEE manifest)
TA stack	32–64 KB
Sealed model weights in TA	50–200 KB compressed surrogate
Power budget (P)	Measured via USB power meter / PMIC; reported as joules/inference

TinyML baseline (PyTorch → TFLite in REE or small C TA) is the reference point; hybrid QML must match or beat it on recall@FPR **within these budgets**, or we report that QML is not justified.